

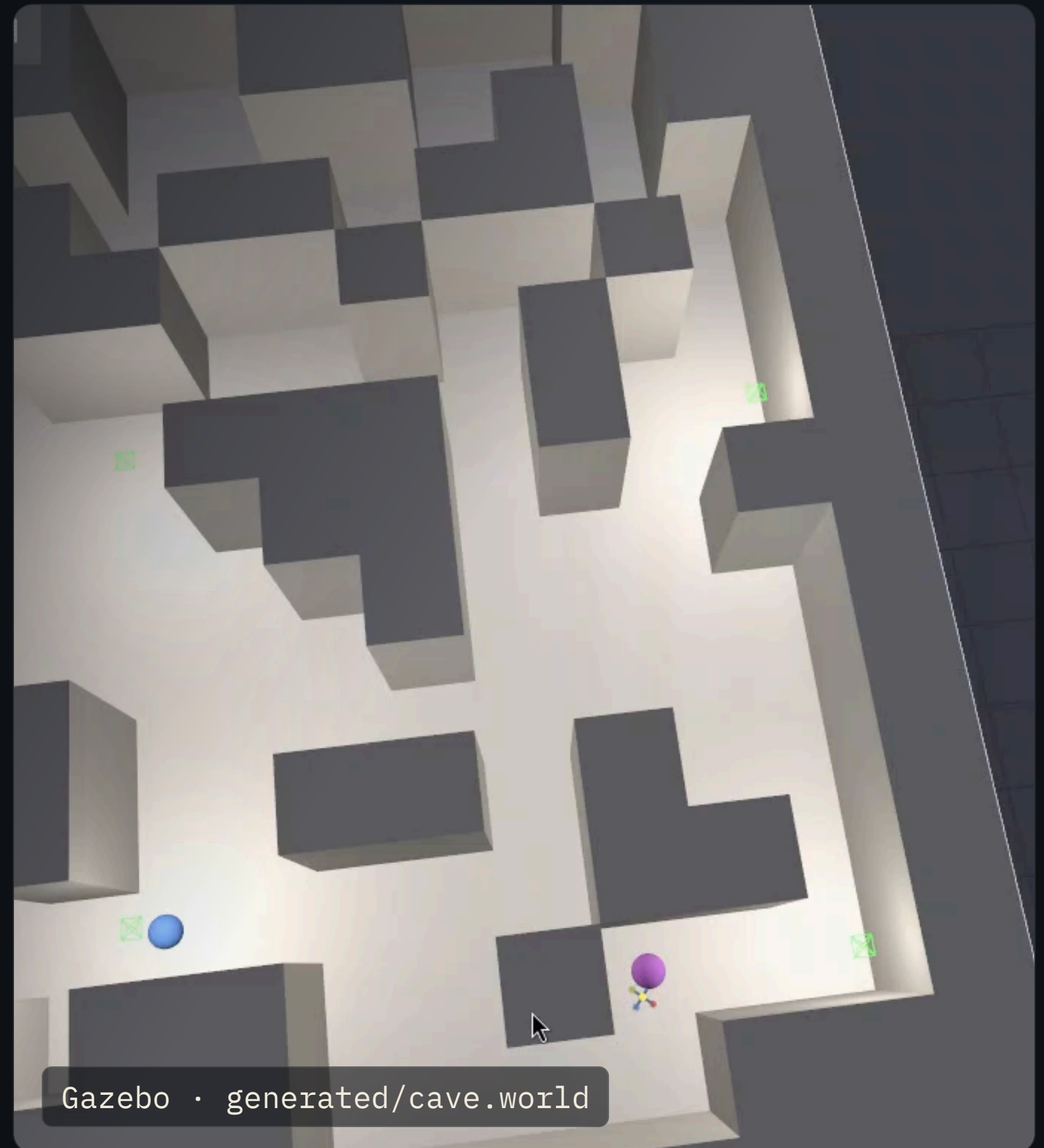
Cave Robot

An autonomous drone that plans, localizes, and maps its way through procedurally generated cave systems — then finds its way back.

Artem Pugachuk
Author

Arbi Kullakshi
Author

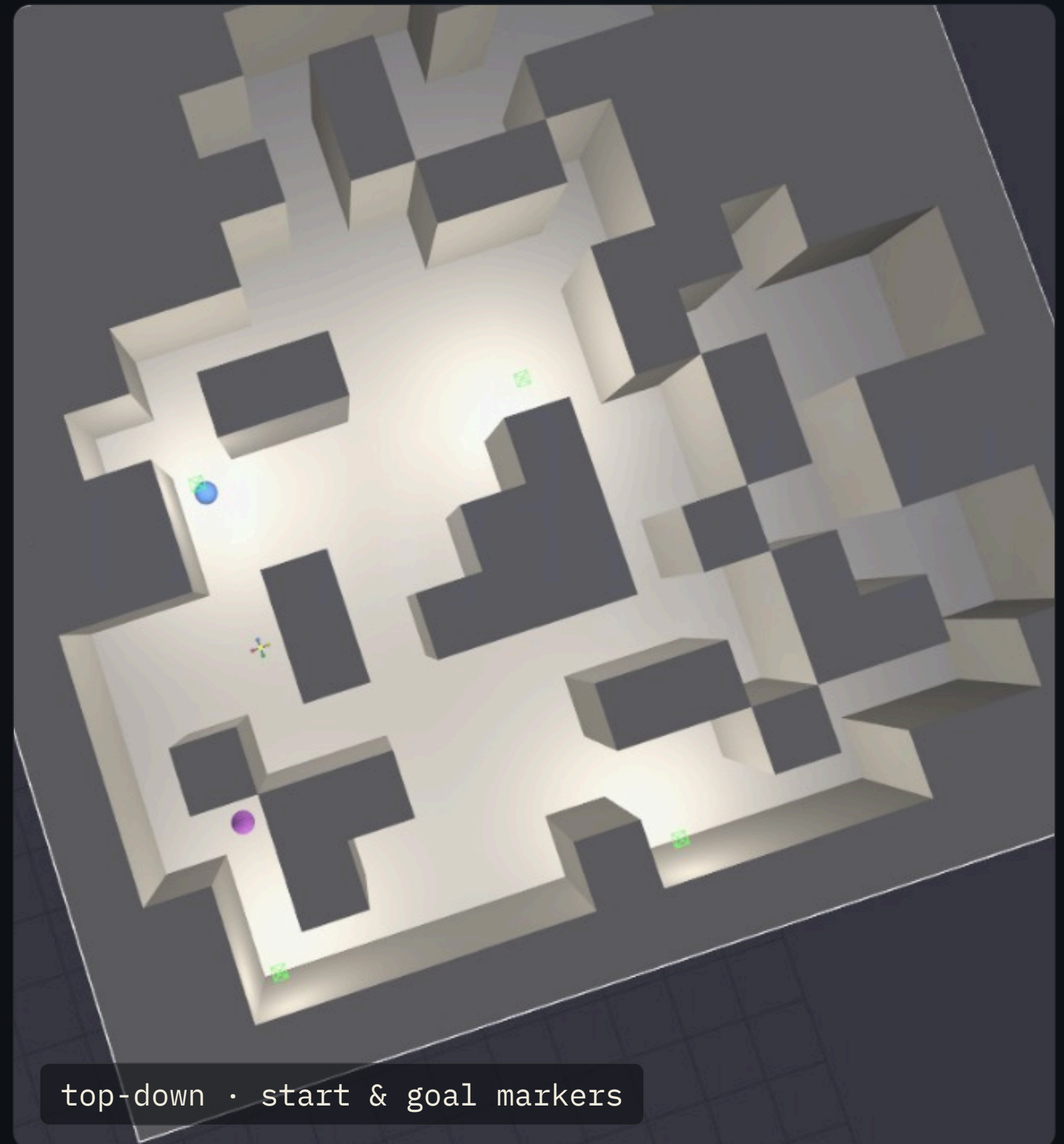
A* · EKF · SLAM
Rust · ROS 2 · Gazebo



Get from A to B in a space full of walls.

- 01 A robot is dropped into an unfamiliar indoor environment and must reach a **start** → **goal** destination.
- 02 It cannot collide with walls, and dead-reckoning drifts — wheel/motor estimates accumulate error over time.
- 03 The space must be **mapped while moving**, so the robot can reason about where it has and hasn't been.

Three coupled sub-problems — **localization**, **mapping**, and **planning** — solved together in one round trip.



top-down · start & goal markers

One mission, two journeys.

① FORWARD • START → END

Plan A* on the **known** clearance-weighted graph. Drive to the goal while LiDAR SLAM records what is actually seen.



② OBSERVE & MAP

An occupancy grid is built from LiDAR scans — the map the robot **earns** by observation, not the ground truth.



③ RETURN • END → START

Re-plan A* on the **SLAM-derived** map and navigate home using only what was learned on the way out.

● START — known graph ● END — earned map

Localization runs continuously via an **Extended Kalman Filter**.

Rust navigation core, bridged to ROS 2 & Gazebo.

NAVIGATION CORE · RUST

pathfinding A* · D* Lite

kalman EKF localization

slam occupancy grid

generator + shared caves · types

SIMULATION BRIDGE · ROS 2

cave_robot_node spawns robot --pipe

▲ stdout JSON → cmd_vel ▲

▼ GPU LiDAR + pose → stdin ▼

ros_gz_bridge topics

Gazebo generated/cave.world

Navigation logic talks to the simulator over a line-delimited JSON pipe via [rclrs](#) — the ROS 2 Rust client library.

Caves grown by cellular automata.

- Each z-slice starts as random noise, then a [birth/survival](#) smoothing rule iterates wall ↔ floor until organic chambers emerge.
- A [connectivity pass](#) guarantees start and end sit in the same traversable region — no unreachable goals.
- The result is exported to [cave.json](#) and compiled into a Gazebo `cave.world` via `cave_to_world`.

```
generator → cave.json → cave_to_world →  
worlds/generated/
```

cellular-automata cave · per z-slice

An Extended Kalman Filter keeps the pose honest.

Predict Motor commands advance the pose by **dead-reckoning** — fast, but error accumulates every step.

Update Gazebo pose measurements **correct** the prediction, weighted by each source's uncertainty.

Fuse The filter linearizes the motion model each step, yielding a single drift-corrected estimate of x , y , θ .

$$\hat{\mathbf{x}}_k = \mathbf{f}(\hat{\mathbf{x}}_{k-1}, \mathbf{u}_k)$$

predict from motion

$$\hat{\mathbf{x}}_k \leftarrow \hat{\mathbf{x}}_k + \mathbf{K}_k (\mathbf{z}_k - \mathbf{h}(\hat{\mathbf{x}}_k))$$

correct with measurement

LiDAR SLAM builds the map it will navigate home on.

- A GPU LiDAR sweeps the cave each step; ray endpoints are folded into an **occupancy grid**.
- Cells accumulate evidence — hits raise occupancy, pass-throughs clear free space — using a **particle-filter** estimate of the robot's track.
- By the time the goal is reached, the forward trip has produced a usable, self-built map of the traversed corridors.



A* on a clearance-weighted grid.

$$f(n) = g(n) + h(n)$$

- g Cost accrued so far — summed **per-cell** weights.
- h Admissible estimate of the remaining distance to goal.

CLEARANCE WEIGHTING

Cells near walls cost more, so the planner prefers routes through open space — paths that keep a safety margin instead of scraping corners.

3-D CARDINAL GRAPH

Search runs on a 3-D cardinal-direction grid; D* Lite is available for incremental replanning when the map changes.

Navigating home on a map it earned.

FORWARD TRIP USED...

The **ground-truth** clearance-weighted graph — the cave as it truly is, known in advance.

VS

RETURN TRIP USES...

The **SLAM-derived** occupancy grid — only the corridors actually observed on the way out.

The return is the real test: A* re-plans **end** → **start** over the self-built map, proving the robot can act on its own perception — not a map handed to it.

A full scene, spun up from one command.



cave_walls generated geometry

cave_robot LiDAR + pose publisher

start_marker blue sphere

end_marker violet sphere

fill_light + cave_light_0-3 scene lighting

The round trip closes.

A → B → A

Complete round trip: planned out on the known graph, returned on the earned map.

2 maps

Ground-truth graph for the forward leg; SLAM occupancy grid for the return leg.

0 walls

Clearance-weighted A* favours open corridors, keeping a margin from obstacles.

EKF fusion keeps the estimated pose aligned with Gazebo ground truth across both legs, so the planner and the map stay consistent end-to-end.

One toolchain, four steps.

```
# enter the reproducible container
$ distrobox enter cave-robot-ros-two

$ just build # cargo + colcon
$ just generate # cave.json
$ just generate-world # SDF world
$ just bringup # Gazebo + bridge + robot
```

generator	Cellular-automata caves, connectivity, JSON export
robot	A*, EKF, SLAM loop, pipe protocol
pathfinding	A* & D* Lite on a 3-D cost grid
slam	Particle-filter occupancy mapping
kalman	EKF dead-reckoning + pose fusion
cave_robot_node	ROS 2 bridge: LiDAR ⇔ cmd_vel

What's next.

LIMITATIONS

- Forward planning assumes the full map is known in advance.
- Localization leans on Gazebo's ground-truth pose alongside dead-reckoning.
- Validated in simulation only — no real hardware or sensor noise model yet.

FUTURE WORK

- Plan the forward leg on the SLAM map too — fully online exploration.
- Replace pose measurements with scan-matching for hardware realism.
- Use D* Lite for live replanning as the occupancy grid updates.

Plan. Localize. Map. Return.

Cave Robot ties classical robotics — A^* , the **Extended Kalman Filter**, and **LiDAR SLAM** — into a single indoor-navigation round trip, implemented in Rust and demonstrated live in ROS 2 & Gazebo.

Artem Pugachuk

Indoor Mobile Robot Navigation

Arbi Kullakshi

`github.com/somethim/cave-robot`